Projet Algorithmique: L’Akinator des pays européens

Présenté par Raphael Bernard & Aissatou Diallo

Sommaire

01 - Intro

02 - Répartition

03 - Besoins

04 - Fonctionnalité

05 Cas d'utilisation

06 - Architecture

07- Arbre de decision

08 - Demonstration

09 - Ameliorations

10 - Difficultés

Introduction

LE PROJET

- L'objectif:
  - créer une version simplifiée d'Akinator
  - Si le personnage ou l'objet n'est pas trouvé, le programme ajoute automatiquement ce dernier à partir d'une question caractéristique.
  - À la fin de la session, les nouvelles informations sont enregistrées dans un fichier.

AKINATOR DES PAYS EUROPÉENS

- Principe:
  - Akinator qui devine le pays européen que l'utilisateur a en tête
  - Utilise un arbre de décisions de forme .txt
  - Interface graphique utilisant GfxLib
  - Fonctionnalité de sauvegarde des informations saisies

Répartition des tâches

Raphael - Codage et implémentation de l’arbre de décision

Aissatou - Graphique utilisant GfxLib pour l’Akinator et la version future

03 - Etude des besoins

- ENVIRONNEMENT DE DÉVELOPPEMENT LINUX
  - MACHINE VIRTUELLE
  - LANGAGE C
  - BIBLIOTHÈQUE GRAPHIQUE GFXLIB

- LE CODE
  - UN ARBRE DE DÉCISION FONCTIONNEL
  - MAKEFILE
  - GESTION DE FICHIERS POUR LA SAUVEGARDE

- INTERFACE GRAPHIQUE

04 - Fonctionnalité

Quelles sont les fonctionnalités de notre code ?

- Permet de répondre à des questions selon un arbre de décision
- Sauvegarde des questions et réponses de la session dans un fichier pays.txt
- Permet de ‘jouer’ à Akinator grâce à une interface graphique avec des boutons
- Le code devine le pays européen que l’utilisateur a en tête
- Si le pays n’est pas le bon; Option de saisir le pays non-trouvé qui sera ajouté à la base de données
- Option de rejouer autant de fois désirées sans ré-executer le code

05 - Cas d’utilisation

Un exemple concret…

Un utilisateur pense à l’Espagne, et le programme pose des questions pour deviner ce pays.

1. Le code pose la question ‘racine’: Ce pays est-il membre de l’union européenne ?
2. En fonction de la réponse (oui = 1, non = 0), le code va parcourir l’arbre de décision et poser la question de forme question:reponse (reponse = 0 ou 1).
3. Ce procédé est répété jusqu’à une question caractéristique, pour l’Espagne: Ce pays possède-t-il l’arme nucléaire ?10101
4. Akinator finis par deviner le bon pays, l’Espagne, puis demandera à l’utilisateur s’il veut rejouer.
5. Les questions + réponses seront sauvegardées dans pays.txt

05 - Cas d’utilisation

Un autre exemple ...

Un utilisateur pense à un pays pas en europe, mexique par exemple, et le programme pose des questions pour deviner ce pays.

1. Le code pose la question ‘racine’: Ce pays est-il membre de l’union européenne ?
2. En fonction de la réponse (oui = 1, non = 0), le code va parcourir l’arbre de décision et poser la question de forme question:reponse (reponse = 0 ou 1).
3. Ce procédé est répété jusqu’à une question caractéristique, et Akinator va deviner le mauvais pays car l’utilisateur n’a pas lu les consignes et a choisi un pays inconnu au code.
4. Akinator demande donc a l’utilisateur quel pays il avait en tête, et enregistre ce nouveau pays dans pays.txt

06 - L’architecture

Les bibliothèques nécessaires: stdlib, stdio, string, GfxLib, BmpLib, ESLib

```
1 #include <stdlib.h> // gestion de la mémoire dynamique, conversions, etc.
2 #include <stdio.h> // opérations d'entrée/sortie (comme printf et fopen).
3 #include <string.h> // manipulation de chaînes de caractères (comme strcpy et strcmp).
4
5 #include "GfxLib.h" // bibliothèque graphique GfxLib
6 #include "BmpLib.h" // la manipulation des images BMP.
7 #include "ESLib.h" // gestion des événements.
```

06 - L’architecture

Les prototypes utilisées

```
void gestionEvenement(EvenementGfx evenement);
Noeud* creerNoeud(char *question);
void construireArbre(Noeud *racine, FILE *f_questions);
void obtenirQuestionSuivante(Noeud *racine, char *chemin, char *questionTexte);
void sauvegarderPays(const char *pays);
void sauvegarderQuestion(const char *chemin, const char *question);
```

06 - L’architecture

Les différents modules

- **Module d’initialisation et gestion principale:**
  - Ce module inclut les fonctions main () et gestionEvenement () et permet d’initialiser les ressources necessaires.

06 - L’architecture

Les différents modules

Module Arbre de decision:

- les fonctions: creerNoeud(),construireArbre() et obtenirQuestionSuivante() et permet de gérer la creation et la manipulation de l’arbre de decision.

```
Noeud* creerNoeud(char *question);
void construireArbre(Noeud *racine, FILE *f_questions);
void obtenirQuestionSuivante(Noeud *racine, char *chemin, char *questionTexte);
```

L’arbre de décision:

- dans un fichier questions.txt
- de forme question:reponse
- 0 = non, 1 = oui

```
Ce pays est-il un pays slave?:0
Ce pays faisait il partit du bloc de l'Est pendant la guerre froide (au moins partiellement) ?:1
Ce pays est-il une micro-nation?:00
Ce pays est-il dans les Balkans?:01
Ce pays borde-t-il la mer Méditerranée?:10
Ce pays borde-t-il la mer Baltique:11
Ce pays est-il plus au Nord que la ville de Lille?:000
Ce pays est-il enclavé dans le territoire Italien?:001
Ce pays est-il frontalier de la Lituanie?:010
Ce pays est-il frontalier de l'Albanie?:011
Ce pays est-il voisin avec l'Allemagne?:100
Ce pays est-il une ile?:101
Ce pays a-t-il accès à un littoral?:110 Ce pays est-il un pays balte?:111
Ce pays est-il enclavé?:0000
Ce pays est-il situé sur une ile?:0001
Ce pays possède-t-il une frontière avec la France?:0010
Ce pays est-il une théocratie?:0011
```

```c
Noeud* creerNoeud(char *question) {
    Noeud *nouveau = (Noeud *)malloc(sizeof(Noeud));
    strcpy(nouveau->question, question);
    nouveau->oui = NULL;
    nouveau->non = NULL;
    return nouveau;
}
void construireArbre(Noeud *racine, FILE *f_questions) {
    char line[256];
    while (fgets(line, sizeof(line), f_questions) != NULL) {
        char *pos = strchr(line, ':');
        if (pos != NULL) {
            *pos = '\0';
            char *chemin = pos + 1;
            chemin[strcspn(chemin, "\n")] = '\0';  // retirer caractère de la nouvelle ligne
            Noeud *noeud = racine;
            while (*chemin) {
                if (*chemin == '1') {
                    if (noeud->oui == NULL) {
                        noeud->oui = creerNoeud(line);
                    }
                    noeud = noeud->oui;
                } else if (*chemin == '0') {
                    if (noeud->non == NULL) {
                        noeud->non = creerNoeud(line);
                    }
                    noeud = noeud->non;
                }
                chemin++;
            }
        }
    }
}
void obtenirQuestionSuivante(Noeud *racine, char *chemin, char *questionTexte) {
    Noeud *noeud = racine;
    while (*chemin) {
        if (*chemin == '1') {
            noeud = noeud->oui;
        } else if (*chemin == '0') {
            noeud = noeud->non;
        }
        chemin++;
    }
    strcpy(questionTexte, noeud->question);
}
```

06 - L’architecture

Les différents modules

Module gestion des événements:

- Ce module inclut la fonction gestionEvenement() et permet de gerer les événements de l’utilisateur comme les clics de souris et les saisies au clavier.

```
void gestionEvenement(EvenementGfx evenement);
```

07 - Arbre d’appel des fonctions

les fonctions appelées directement depuis main

1. initialiseGfx(argc, argv)
2. prepareFenetreGraphique("Projet Pays", LargeurFenetre, HauteurFenetre)
3. fopen("questions.txt", "r+")
4. fopen("pays.txt", "a+")
5. creerNoeud("Ce pays est-il membre de l'Union Européenne ?")
6. construireArbre(racine, f_questions)
7. obtenirQuestionSuivante(racine, chemin, questionTexte)
8. lanceBoucleEvenements()
9. fclose(f_questions)
10. fclose(f_reponses)
11. libereDonneesImageRGB(&akinatorImage)

07 - Arbre d’appel des fonctions

Les fonctions suivantes sont appelées à travers gestionEvenement, qui est une fonction dédiée à la gestion des événements graphiques :

- effaceFenetre
- ecrisImage
- couleurCourante
- rectangle
- afficheChaine
- caractereClavier
- strncat
- abscisseSouris
- ordonneeSouris
- sauvegarderPays
- sauvegarderQuestion

08-Démonstration

v

Pour lancer le code il faut taper :
make pour compiler et
./**pays** pour exécuter .

09-Améliorations

- **Fonctionnalité ajoutée:** Synthèse vocale pour améliorer l'accesibilité aux personnes ayant des hadicaps.
- **Utilisation de eSpeak**

```c
void gestionEvenement(EvenementGfx evenement);
Noeud* creerNoeud(char *question);
void construireArbre(Noeud *racine, FILE *f_questions);
void obtenirQuestionSuivante(Noeud *racine, char *chemin, char *questionTexte);
void sauvegarderPays(const char *pays);
void speak(const char *text);
// Fonction pour lire le texte à haute voix en français avec eSpeak
void speak(const char *text) {
    char command[512];
    snprintf(command, sizeof(command), "espeak -v fr \"%s\"", text);  // Utiliser la voix française
    system(command);
}
```

10-Difficultés

- **Problème:** On a eu du mal à parcourir l'arbre de décision, le programme lisait le fichier questions.txt.
- **Solution:**
  - Pour résoudre cela on a fait une fonction qui utilise une boucle pour séparer la question du chemin en lisant chaque ligne du fichier.
- **Problème:** Avec la version future ,la lecture du texte était en continu tant que l'utilisateur n'appuyait pas sur un bouton.
- **Solution:**
  - Pour y remédier on a utilisé une variable de type booléen qui sert de marqueur pour indiquer si un texte spécifique a déjà été lu à haute voix par le programme.

Merci de votre attention !